

```
import matplotlib.pyplot as plt
import numpy as np
import os
import PIL
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.python.keras.layers import Dense, Flatten
from tensorflow.keras.models import Sequential
```

✓ Preparing The Data

```
import pathlib
flowers_url = "https://storage.googleapis.com/download.tensorflow.org/example_images/flower_photos.tgz"
flowers_data = tf.keras.utils.get_file('flower_photos', origin=flowers_url, untar=True)
flowers_data = pathlib.Path(flowers_data)
```

⤴ Downloading data from https://storage.googleapis.com/download.tensorflow.org/example_images/flower_photos.tgz
228813984/228813984 ————— 2s 0us/step

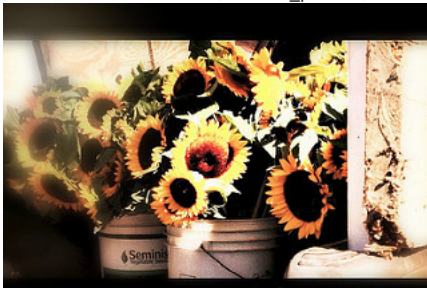
```
print(flowers_data)
```

⤴ /root/.keras/datasets/flower_photos

Start coding or [generate](#) with AI.

```
all_sunflowers = list(flowers_data.glob('sunflowers/*'))
print(all_sunflowers[1])
PIL.Image.open(str(all_sunflowers[1]))
```

⤴ /root/.keras/datasets/flower_photos/sunflowers/13648603305_1268eda8b7_n.jpg



```
height,width=180,180
```

```
training_batch_size=32
```

```
train_set = tf.keras.preprocessing.image_dataset_from_directory(
    flowers_data,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=(height,width),
    batch_size=training_batch_size)
```

⤴ Found 3670 files belonging to 5 classes.
Using 2936 files for training.

```
validation_set = tf.keras.preprocessing.image_dataset_from_directory(
    flowers_data,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(height, width),
    batch_size=training_batch_size)
```

Found 3670 files belonging to 5 classes.
Using 734 files for validation.

```
image_cat = train_set.class_names
print(image_cat)
```

```
['daisy', 'dandelion', 'roses', 'sunflowers', 'tulips']
```

✓ Training The Model

```
dnn_model = Sequential()
```

```
imported_model= tf.keras.applications.ResNet50(include_top=False,
        input_shape=(180,180,3),
        pooling='avg',classes=5,
        weights='imagenet')
```

```
for layer in imported_model.layers:
    layer.trainable=False
```

```
dnn_model.add(imported_model)
dnn_model.add(keras.layers.Flatten())
dnn_model.add(keras.layers.Dense(512, activation='relu'))
dnn_model.add(keras.layers.Dense(5, activation='softmax'))
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_kernels_no
94765736/94765736 ————— 0s 0us/step

```
from tensorflow.keras.optimizers import Adam
dnn_model.compile(optimizer=Adam(),loss='sparse_categorical_crossentropy',metrics=['accuracy'])
```

```
history = dnn_model.fit(
    train_set,
    validation_data=validation_set,
    epochs=10
)
```

```
Epoch 1/10
92/92 ————— 34s 238ms/step - accuracy: 0.6676 - loss: 1.1745 - val_accuracy: 0.8501 - val_loss: 0.4629
Epoch 2/10
92/92 ————— 20s 74ms/step - accuracy: 0.8991 - loss: 0.2880 - val_accuracy: 0.8638 - val_loss: 0.3774
Epoch 3/10
92/92 ————— 11s 84ms/step - accuracy: 0.9291 - loss: 0.1879 - val_accuracy: 0.8569 - val_loss: 0.3676
Epoch 4/10
92/92 ————— 10s 86ms/step - accuracy: 0.9632 - loss: 0.0983 - val_accuracy: 0.8760 - val_loss: 0.3703
Epoch 5/10
92/92 ————— 10s 79ms/step - accuracy: 0.9866 - loss: 0.0583 - val_accuracy: 0.8787 - val_loss: 0.3852
Epoch 6/10
92/92 ————— 9s 94ms/step - accuracy: 0.9935 - loss: 0.0309 - val_accuracy: 0.8842 - val_loss: 0.3839
Epoch 7/10
92/92 ————— 8s 87ms/step - accuracy: 0.9995 - loss: 0.0143 - val_accuracy: 0.8869 - val_loss: 0.4012
Epoch 8/10
92/92 ————— 10s 83ms/step - accuracy: 1.0000 - loss: 0.0067 - val_accuracy: 0.8924 - val_loss: 0.3964
Epoch 9/10
92/92 ————— 9s 93ms/step - accuracy: 1.0000 - loss: 0.0049 - val_accuracy: 0.8801 - val_loss: 0.4153
Epoch 10/10
92/92 ————— 8s 89ms/step - accuracy: 1.0000 - loss: 0.0047 - val_accuracy: 0.8883 - val_loss: 0.4020
```

```
dnn_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
resnet50 (Functional)	(None, 2048)	23,587,712
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 512)	1,049,088
dense_1 (Dense)	(None, 5)	2,565

Total params: 26,742,673 (102.02 MB)

Trainable params: 1,051,653 (4.01 MB)

Non-trainable params: 23,587,712 (92.00 MB)

✓ Making Predictions

```
import cv2
sample_image=cv2.imread(str(all_sunflowers[1]))
sample_image_resized= cv2.resize(sample_image, (height, width))
sample_image=np.expand_dims(sample_image_resized,axis=0)
print(sample_image.shape)
```

(1, 180, 180, 3)

```
model_pred=dnn_model.predict(sample_image)
print(model_pred)
```

1/1 4s 4s/step
[[6.6228670e-01 1.6581887e-06 4.9671125e-02 2.4556877e-01 4.2471714e-02]]

```
predicted_class=image_cat[np.argmax(model_pred)]
print("The predicted category is", predicted_class)
```

The predicted category is daisy